

Stakeholder

Who cares · buy/build · sim factory · motion deep dive · risks

Product one-liner

RoboGen is an LLM-orchestrated sim factory that mints task-scene-reward-demo tuples so robots practice novel skills without real-world collection cost.

Tattoo: 0.774/69 · 19/155 fails · 3.13 substeps · Self-BLEU 0.284 · Fig.5 RL-only collapses

Who cares / who does not

Consumers vs non-consumers

Pays / benefits

- Labs needing many novel household skills without teleop armies
- Sim-platform teams wanting auto task+scene+reward layer
- BEHAVIOR-class teams hungry for coverage before scarce gold demos
- Anyone stuck on sim authoring bottleneck (not real diversity)

Non-consumers

- Need certified real demos only (UMI/DROID/OXE poles)
- No sim budget / no LLM API budget
- Require verified sim-to-real today (paper leaves gap open)
- Tabletop pick-place only → GenSim-class may suffice

Buy / build decision tree

Sim authoring vs real diversity bottleneck

1. IF Bottleneck = authoring tasks/rewards/scenes in sim

→ Adopt RoboGen-style propose→generate→learn

2. IF Bottleneck = real visual / tactile diversity

→ Still fund UMI / DROID / OXE; RoboGen = sim prior / bootstrap

3. IF Need articulated / soft / loco beyond pick-place

→ RoboGen value > GenSim-class tabletop code-gen

4. IF Ship endless mint into training mixes

→ Budget human audit ~12% fail (19/155) + verification

Opposite poles — RoboGen vs UMI / DROID / OXE

Sim generative factory vs real collection

RoboGen

LLM sim factory

Task+scene+reward
+ MP/RL/traj-opt
Self-BLEU 0.284

UMI

Human + gripper

Wild demos
no robot at collect
~\$371

DROID

Franka in wild

76k / 564 scenes
real buildings

OXE

Multi-lab real

22 robots soup
community scale

Fig. 2 — generative sim factory

A propose → B scene → C supervision → D skill learn

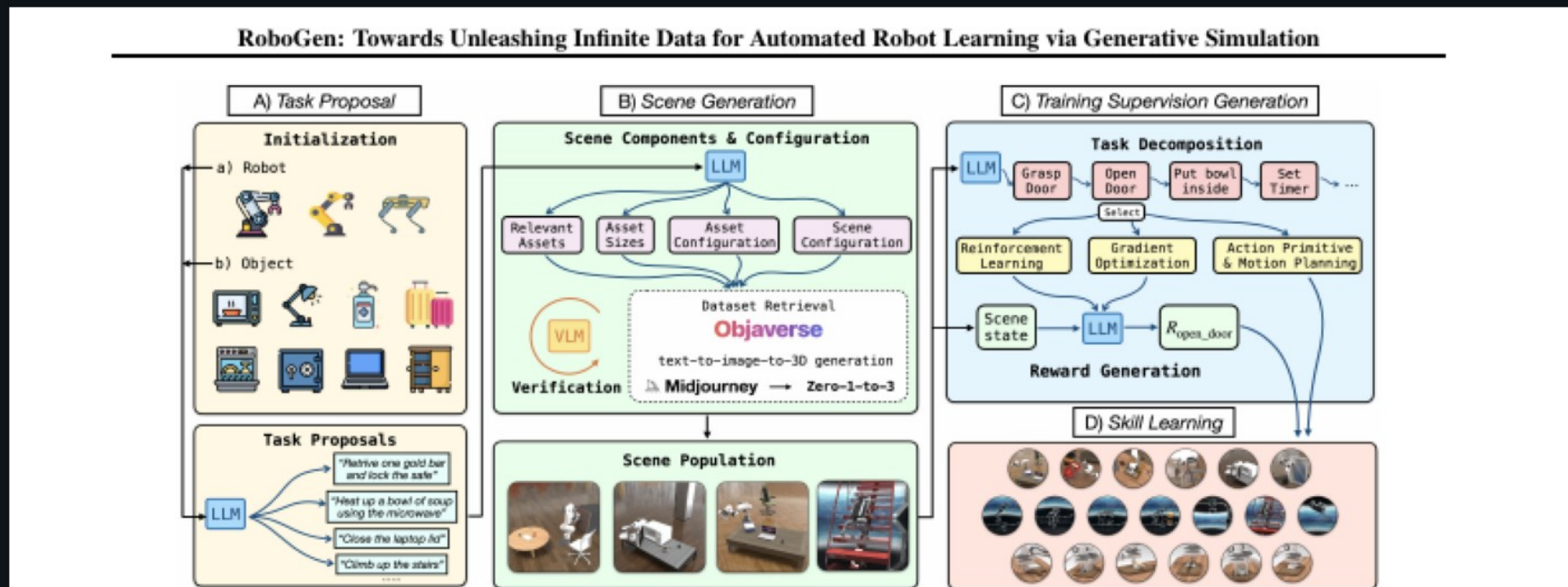


Figure 2: RoboGen consists of the following stages: A) task proposal, B) scene generation, C) training supervision generation, and D) skill learning with generated information.

from its URDF file, and 3) semantic annotations of the links in the object's articulation, e.g., which link corresponds to the door in a sampled microwave. These information are provided by the PartNetMobility dataset. Additionally, we include one example input-output pair in the prompt. We

3.2. Scene Generation

Once a task proposal is obtained, RoboGen then generates a corresponding scene for solving the task by populating the environment with a number of relevant and necessary

Headline metrics investors / PMs pin

Success · validity · diversity · modality splits

0.774

mean SR · 69 tasks

0.745

manip · n=50

~0.886

soft · n=7

~0.833

loco · n=12

19/155

~12% audit fail

0.284

Self-BLEU best

3.13

avg substeps

Fig. 5

MP critical

Motion / skill decomp — algo menu

Competence = decomp + choose, NOT end-to-end LLM policy

SAC RL

Contact-rich /
continuous /
loco knobs

1M steps/sub
lr $3e-4$ · [256]³

BIT* MP

Approach / grasp
/ release paths

OMPL · 0.03 m
pre-contact

Adam traj

Soft-body
shaping (EMD)

300 steps
lr 0.05

CEM loco

Legged gaits
GT dynamics

horizon 150
fs 4

Long-horizon glue

Sequential subtasks · N=8 best-reward end-state → next init · avg 3.13 substeps (~1.5 RL + ~1.63 MP)

Fig. 6 — substep structure is the product

Avg 3.13 · ~1.5 RL · ~1.63 MP · up to 8-10 on long tasks

RoboGen: Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation

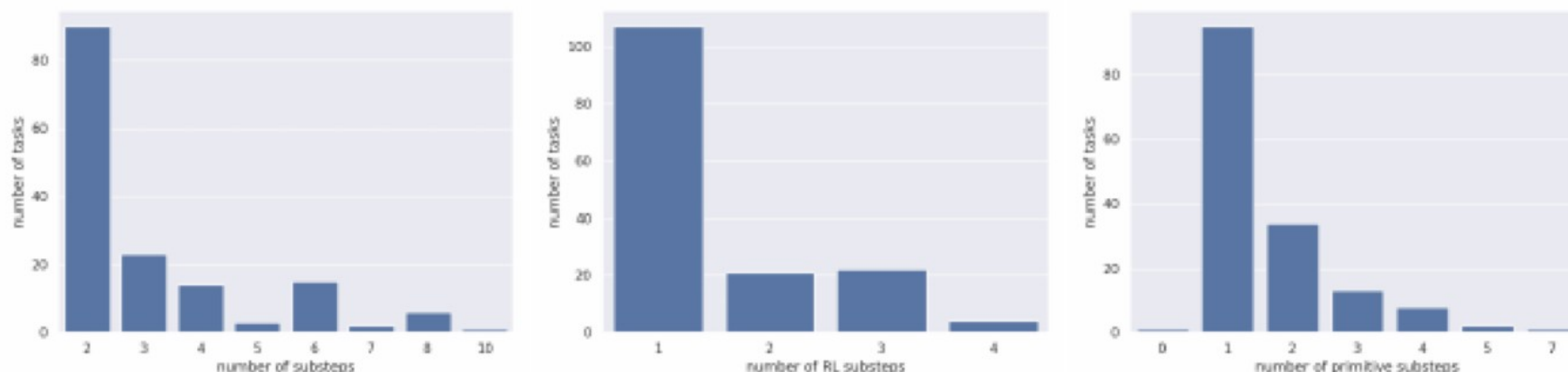


Figure 6: Left: The distribution of number of substeps for the generated rigid and articulated object manipulation tasks in Table 2. The average number of substeps is 3.13. Middle: The distribution of number of substeps that need to be solved using RL for the generated tasks. The average number of RL substeps is 1.5. Right: The distribution of number of substeps that need to be solved using motion planning based primitives for the generated tasks. The average number of such kind of substeps is 1.63. Regarding duration for solving the task: if the task’s subgoals can all be solved via planning, typically each task can be solved within 10 minutes. If certain subgoals require RL to solve, it usually takes around 2-3 hours for each RL-necessary step, and the total duration thus depends on both the number and nature of the subtasks. Taking these into account, a task typically takes 4-5 hours on average. This is done using 8 threads of a CPU running at 2.5Ghz, meaning that each single node in a cluster with a 32-core (64 threads) CPU could run 8 jobs in parallel at the same time.

Fig. 5 — RL-only collapses on articulated

MP primitives are critical · hybrid wins on 12 tasks

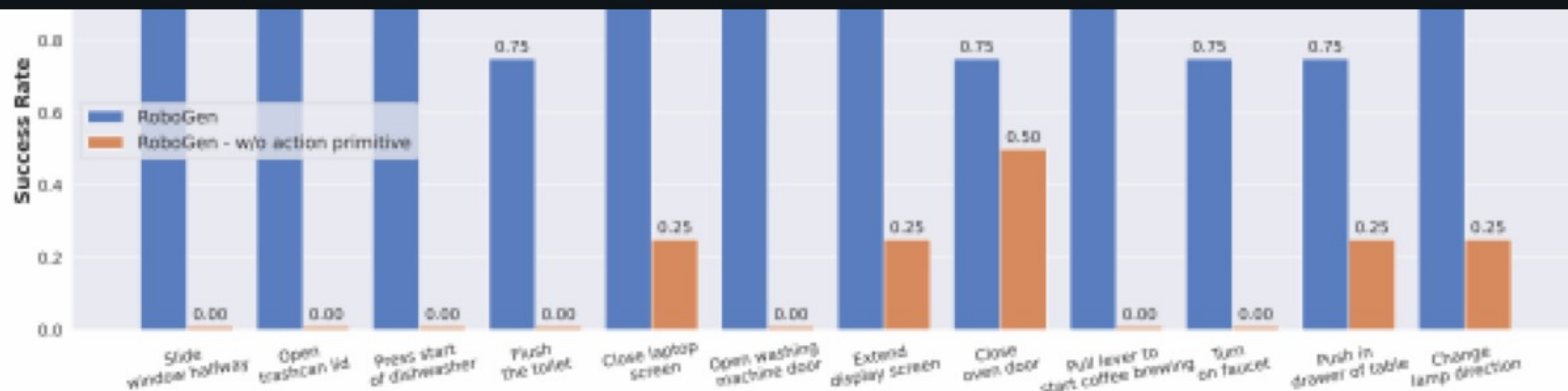


Figure 5: Among 12 articulated object manipulation tasks, the success rate decreases drastically if only RL is used for skill learning.

tion tasks, meanwhile leverage more diverse assets retrieved from open-world databases such as Objaverse, resulting in much higher diversity in both task semantics and scene images. We also provide the full list of generated tasks in

judge whether the joint angle value 0 corresponds to the door being opened or closed. 3) failure to find matched assets for tasks that require extremely precise spatial relationships, which is hard to retrieve from open-world databases and tasks

Self-BLEU 0.284 · wins all ↓ metrics at ~100-task scale

0.284

vs Beh-100 0.299

0.165

vs 0.210

0.193

vs 0.389

0.762

vs 0.833

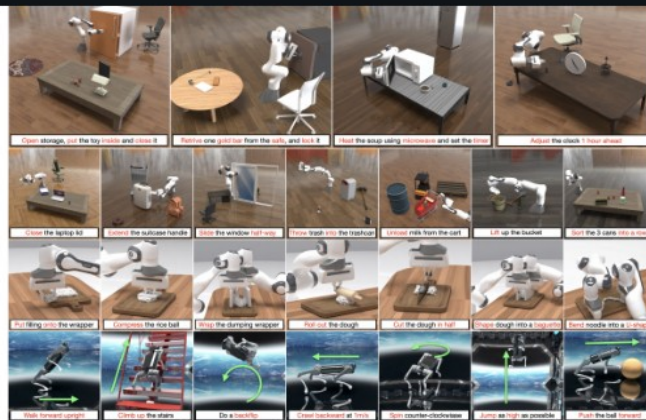


Figure 1: 25 example tasks generated and corresponding skills learned by RoboGen. Readers are encouraged to visit our project website for the diverse set of tasks and skills RoboGen can produce.

ties (Touvron et al., 2023; Driess et al., 2023; OpenAI, 2023; attempts to transfer the extensive and versatile knowledge

Budget the human audit (~12% fail)

19/155 = 13 scene + 6 reward/decomp · do not ship unattended

13 / 155

scene fails

6 / 155

reward/decomp fails

~12%

total fail rate

- Scene fails: missing functionality, joint↔semantic map, precision pairs (stapler↔staples).
- Reward fails: undefined vars, inverted fold/unfold, continuous rhythmic motions.
- Wall-clock: planning ≤ 10 min; each RL subgoal $\sim 2-3$ h; typical task 4-5 h (CPU-heavy).
- Stakeholder rule: verification + spot audit before demos enter any VLA mix.

Risks — API / Genesis / sim2real

API cost & drift
Minimal human $\neq$ minimal compute or \$

GPT-4 / Gemini / Midjourney closed,
moving targets

Genesis opacity

Internal soft-body sim at pub;
public PyBullet subset

Silent reward bugs

Inverted semantics pass syntax; need
audits

Sim-to-real tax

\$5 admits gap; no transfer table in
paper

Verification scale

Human video SR; auto skill verify
still open

Endless query \$

API + CPU parallel; not free
infinite data

Numbers tattoo — pin these

Every deck · Ablation Bot MISSING (paper extract)

0.284

Self-BLEU (best)

0.165

SentBert ↓

0.193

Scene ViT ↓

0.762

Scene CLIP ↓

0.774

success / 69

19 / 155

validity fails

3.13

avg substeps

Fig. 5

RL-only collapses

0.745

manip SR (n=50)

~0.886

soft SR (n=7)

~0.833

loco SR (n=12)

~1.5+1.63

RL + MP avg

Stakeholder — close

Takeaways + exam bite

1. Buy RoboGen-style factory when sim authoring is the bottleneck.
2. Keep UMI/DROID/OXE for real diversity; RoboGen complements, does not replace.
3. MP primitives + algo select are the product — Fig. 5 collapses without them.
4. Budget $\sim 12\%$ human audit; pin $0.774/69 \cdot 0.284$ Self-BLEU $\cdot 3.13$ substeps.

EXAM BITE

Competence = decomp + choose MP/RL/traj-opt per sub-skill — NOT end-to-end LLM policy.